

FinTrust Portfolio and Capstone Plan

A clear guide to completed portfolio work, the AWS Console build, and the final capstone deliverables

Learner: Nompilo Eugenia Mchunu

Programme: Cloud to Solutions Accelerator

Project: Cloud Based Transaction Insights and Risk Monitoring System

This report uses the completed Weeks 1 to 12 portfolio work together with the supplied VPC and capstone brief. It is designed to show what is already prepared, what must still be built, and what evidence is needed before the live presentation.

How to read this report

Repository work means code, SQL, diagrams, notes, tests, and reflections that are already present in GitHub. AWS Console work means a real cloud resource or result that must be verified in the learner's own AWS account. A design document is useful evidence of planning, but it is not evidence that a service is deployed.

The immediate goal

Build one integrated, safe, Free Tier FinTrust system during Weeks 13 and 14. It must ingest sample transactions, score risk, store records, expose a protected API, alert the compliance team, and create a monthly report. Week 16 is for a live demonstration and business presentation.

1. Portfolio work already prepared

Weeks	Work recorded	Capstone value
1 and 2	AWS foundations, IAM, EC2, resilience, SQL queries, joins, aggregates, and Python basics	Least privilege access, data queries, and good Git practice
3	S3 design, bucket security, lifecycle choices, CSV and JSON processing, and error handling	Storage patterns for raw, processed, quarantine, and report data
4	Relational modelling, RDS, DynamoDB, database selection, Python packages, and ETL	Transaction ledger, event store, and package structure
5	Multi AZ networking, security groups, Route 53, and CloudFront	Private database placement and controlled database access
6	Permission boundaries, CloudTrail, CloudWatch, Config, incident response, and SQL analytics	Governance, logging, alarms, and security decisions
7	SQS, SNS, EventBridge, Lambda, APIs, and CloudFormation	Event processing, alerts, API endpoints, and monthly scheduling
8	S3 data lake, Athena, Glue, Kinesis, analytics, and intelligent services	Data platform and analytics context
9 and 10	Cost control, migration planning, DMS, DataSync, recovery, and packaged migration helpers	Free Tier decisions and legacy migration strategy
11 and 12	Well Architected review, advanced SQL, resilient Python, Cost Explorer, SCPs, scheduling, and tuning	Monitoring, governance, reporting, and final quality checks

The weekly portfolio is a strong foundation. The capstone is separate work that combines these skills into one working system. Earlier weekly exercises should be reused only when they fit the capstone design and have been tested again.

2. What the capstone must do

FinTrust needs an event driven platform for transaction risk monitoring. A safe sample CSV is uploaded to S3. An ingest function validates each row and writes the approved information to the relational ledger and event store. Transactions pass through an ordered SQS FIFO queue for risk scoring. High risk activity can send an email alert. The compliance team uses a protected API, while a scheduled function creates monthly CSV and JSON reports.

Stage	Outcome
1. Upload	A sample transaction CSV reaches the raw S3 bucket.
2. Validate	The ingest Lambda checks fields and moves failed rows to the quarantine path.
3. Store	Valid transactions are written to RDS PostgreSQL and DynamoDB.
4. Score	The FIFO queue triggers risk scoring and preserves per account ordering.
5. Alert	High risk activity invokes the alert Lambda and a safe SES test email.
6. Query and report	API Gateway exposes the compliance endpoints; EventBridge triggers the monthly report.

3. AWS services for the capstone

Service	Use in FinTrust	Evidence to collect
S3	Raw upload, processed data, quarantine data, and monthly reports	Bucket settings, safe test upload, and generated report
Lambda	Ingest, score, alert, query, and report functions	Five functions, triggers, logs, and a test run
SQS FIFO and DLQ	Ordered transaction processing and failed message recovery	Queue, DLQ, and event source mapping
RDS PostgreSQL	Normalised transaction ledger	Private instance, schema, and controlled connection
DynamoDB	Risk scores, session tokens, and event log	Key design and safe test items
API Gateway	Protected compliance and fraud REST API	Deployed stage, API key, and working requests
SES	Email alert for high risk activity	Verified sandbox recipient and safe alert test
EventBridge	Monthly report schedule	Rule, Lambda target, and manual test
CloudWatch and X Ray	Metrics, alarms, dashboard, logs, and tracing	Dashboard, alarm, structured logs, and trace
IAM, Parameter Store, KMS	Least privilege roles, configuration, and bucket encryption	Role matrix, scoped policies, parameter and encryption settings
VPC	Private RDS networking and Lambda to RDS security group access	Private subnets and PostgreSQL rule from the approved Lambda group

Cost decision to keep

The earlier VPC lab demonstrates a production design with two NAT Gateways. The capstone brief excludes NAT Gateways because they create charges outside the Free Tier. For the capstone, keep RDS private and use the targeted Lambda to RDS security group rule. Do not recreate the NAT Gateways for this build.

4. AWS Console plan

Complete each step only when the resource exists in the correct AWS Region and a safe record has been saved.

Account safety and naming

- Confirm one target Region and check that every required service is available there.
- Check the billing alarm and budget before creating resources. Keep the programme R200 safety threshold.
- Use one FinTrust naming pattern and non personal tags such as Project=FinTrust and Environment=dev.

Access and storage

- Create separate execution roles for ingest, score, alert, query, and report. Scope each policy to the resources it needs.
- Create raw, processed, quarantine, and report S3 buckets. Block public access and enable the required encryption.
- Store configuration in Parameter Store. Keep passwords, keys, and connection strings out of source control and screenshots.

Network and data

- Create the capstone VPC and private database subnets in two Availability Zones.
- Create the RDS security group. Allow port 5432 only from the approved Lambda security group. Do not make the database public.
- Create the Free Tier RDS PostgreSQL instance and DynamoDB table with account_id and tx_timestamp keys.
- Load the reviewed schema and safe seed data after verifying the local SQL files.

Transaction path and API

- Create the FIFO queue and dead letter queue, then configure the S3 and SQS Lambda triggers.
- Deploy the five Lambda functions and connect the expected resources to each role.
- Configure SES sandbox testing, API Gateway endpoints, API key protection, and the EventBridge monthly rule.

Monitoring and proof

- Create CloudWatch metrics for ingestion errors and high risk transaction counts, then add alarms and a dashboard.
- Enable X Ray tracing for the API and Lambda test path.
- Run a safe sample CSV from upload to report. Save evidence of the API response, logs, dashboard, and report output.

5. Repository deliverables still required

Area	Required deliverable
Architecture	PDF architecture diagram showing services, data flow, and security boundaries
Data model	3NF ERD with entities, attributes, primary keys, and foreign keys
SQL	Schema, at least 200 seed transactions, CTE risk report, running balance query, view, and PostgreSQL function or procedure
Python	One fintrust package with config, db, dynamo, models, validators, ingest, score, alert, query, and report modules
Tests	pytest suite with moto fixtures, at least three test files, and an end to end verification script
CI	GitHub Actions: unit tests on every push and integration tests only on the develop branch
Design documents	API design, IAM role matrix, POPIA and lifecycle statement, migration approach, and risk register
Evidence	Console screenshots, API response captures, CloudWatch logs, and a green GitHub Actions run

Quality checks before deployment

- The schema uses primary keys, foreign keys, indexes on account ID and transaction date, and appropriate NOT NULL rules.
- The risk query uses a CTE to find more than three high risk transactions in a rolling 30 day period.
- The monthly summary uses a running balance window function per account.
- The database function changes account status and records the risk event together.
- Boto3 clients are created outside Lambda handlers and database failures roll back before returning an error.
- Tests use mocked AWS resources and never need live credentials or billable resources.

6. Evidence and presentation readiness

Capture	What it should show
API Gateway	Deployed stage, protected routes, and the intended API configuration
Lambda	Five functions with relevant S3, SQS, API, or schedule triggers
RDS	Running status and proof that public access is disabled
CloudWatch	Dashboard, alarms, and structured JSON output from a real invocation
API test	Successful GET /transactions and POST /upload responses using safe sample data
GitHub Actions	A green CI run for the test suite
End to end test	Sample file upload through validation, storage, scoring, and report or alert

Keep every capture safe. Do not show AWS account numbers, access keys, passwords, personal email addresses, or real customer information. Use only sample transaction data in demonstrations.

Build schedule

Week	Focus	Expected result
13 Day 1	Design and foundation	Architecture, API design, Parameter Store configuration, and Lambda roles
13 Day 2	Data layer	RDS schema, seed data, DynamoDB table, helpers, and tests
13 Day 3	Security and storage	Encrypted buckets, scoped roles, S3 trigger, and DLQ
13 Day 4	ETL	Ingest function with validation, storage, pre score, and structured logs
13 Day 5	Read API	Transactions and insights endpoints
14 Days 1 and 2	Write path and visibility	Write endpoints, alerts, metrics, alarms, dashboard, logs, and tracing
14 Days 3 and 4	Quality and reporting	Tests, CI, end to end script, final SQL, report function, and schedule
14 Day 5	Readiness	Full rehearsal, Well Architected review, evidence check, and repository audit

7. Assessment focus

Area	Points	What strong preparation looks like
Cloud architecture and AWS implementation	20	A working deployed system, clear architecture, and least privilege access
SQL and data modelling	15	Runnable schema, data model, seed data, and all analytical SQL artefacts
Python development and quality	15	Complete ten module package using the taught patterns
Testing and CI	10	Meaningful tests, green GitHub Actions, and end to end verification
Data governance and compliance	10	Specific POPIA, lifecycle, migration, and risk controls
Documentation and portfolio	10	Clear instructions, diagrams, evidence, and organised repository
Live demo and technical panel	10	A rehearsed upload to alert story and architecture specific answers
Business presentation	10	A simple explanation of compliance value, fraud risk, resilience, and cost

Final working rule

Only describe a service as deployed when there is real AWS Console or command evidence for it. The portfolio can honestly show the prepared design and code now. The Console build, evidence collection, testing, and rehearsal are what make the capstone complete.